

SpringBoot + Scheduled 定时任务实现

一、概述

使用 SpringBoot 的 `@Scheduled` 注解，结合邮件服务和第三方 API，实现了两个定时任务场景：

- 1. 定点发送邮件 — 每天早上 8 点检查用户生日，匹配则发送生日祝福邮件
- 2. 定点更新数据 — 每 5 分钟调用免费天气 API 拉取实时数据并缓存

二、前置准备

2.1 启用定时任务

在主启动类添加 `@EnableScheduling` 注解：

```
@SpringBootApplication
@EnableScheduling
public class SpringbootWeek10Application {
    // ...
}
```

2.2 注册 RestTemplate（用于调用天气 API）

```
@Configuration
public class RestTemplateConfig {
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

三、任务一：定点发送生日祝福邮件

3.1 实现思路

- 使用 `@Scheduled(cron = "0 0 8 * * ?")` 每天早上 8 点触发
- 维护一个模拟用户列表（含姓名、邮箱、生日）
- 每次执行时获取当天日期 `MonthDay`，遍历用户列表匹配生日
- 匹配成功则调用 `MailService.sendRichMail()` 发送富文本生日祝福 HTML 邮件

3.2 核心代码

```
@Service
public class BirthdayTaskService {

    private final MailService mailService;

    private static final DateTimeFormatter MD_FORMATTER = DateTimeFormatter.ofPattern("MM-dd");

    private static final List<Map<String, String>> USERS = List.of(
        Map.of("name", "张三", "email", "imoyy@qq.com", "birthday", "05-09"),
        Map.of("name", "李四", "email", "imoyy@qq.com", "birthday", "05-09")
    );

    @Scheduled(cron = "0 0 8 * * ?")
    public void sendBirthdayCards() throws MessagingException {
        MonthDay today = MonthDay.from(LocalDate.now());

        for (Map<String, String> user : USERS) {
            MonthDay birthday = MonthDay.parse(user.get("birthday"), MD_FORMATTER);
            if (today.equals(birthday)) {
                String html = ""
                    <!-- 生日祝福 HTML 模板 -->
                    """.formatted(user.get("name"));
                mailService.sendRichMail(user.get("email"), "    生日快乐! ", html);
            }
        }
    }
}
```

```
}  
}  
}
```

3.3 关键点

要点	说明
MonthDay.parse()	需传入 DateTimeFormatter.ofPattern("MM-dd"), 默认格式为 --MM-dd
生日匹配	使用 MonthDay 忽略年份, 只比较月日
邮件发送	复用之前实现的 MailService.sendRichMail()

四、任务二：每 5 分钟更新天气数据

4.1 实现思路

- 使用 @Scheduled(fixedRate = 300000) 每 5 分钟触发一次
- 调用 [Open-Meteo API](#)
- 解析返回的 JSON, 提取温度和风速
- 将结果格式化为字符串缓存在内存中
- 通过 getCachedWeather() 方法对外暴露

4.2 核心代码

```
@Service  
public class WeatherUpdateService {  
  
    private final RestTemplate restTemplate;  
    private String cachedWeather;  
  
    @Scheduled(fixedRate = 300000)  
    public void updateWeather() {  
        String url = "https://api.open-meteo.com/v1/forecast" +  
            "?latitude=39.9042&longitude=116.4074&current_weather=true";  
        try {  
            JsonNode root = restTemplate.getForObject(url, JsonNode.class);  
            JsonNode weather = root.get("current_weather");  
            double temperature = weather.get("temperature").asDouble();  
            double windSpeed = weather.get("windspeed").asDouble();  
            cachedWeather = String.format("[%s] 北京 - 温度: %.1f°C, 风速: %.1f km/h",  
                LocalDateTime.now().format(  
                    DateTimeFormatter.ofPattern("HH:mm:ss")),  
                temperature, windSpeed);  
            System.out.println("天气已更新: " + cachedWeather);  
        } catch (Exception e) {  
            System.err.println("获取天气数据失败: " + e.getMessage());  
        }  
    }  
  
    public String getCachedWeather() {  
        return cachedWeather;  
    }  
}
```

4.3 关键点

要点	说明
API 选择	Open-Meteo 免费开源, 无需 API Key, 支持全球天气
fixedRate	每 300 秒执行一次, 不考虑上次任务是否完成 (该场景下适用)
异常处理	网络请求加入 try-catch, 避免因单次失败导致定时任务中断

五、@Scheduled 三种模式对比

模式	示例	说明
fixedDelay	@Scheduled(fixedDelay = 5000)	上次执行结束后等待 5 秒
fixedRate	@Scheduled(fixedRate = 6000)	每 6 秒执行一次, 不考虑上次是否完成
cron	@Scheduled(cron = "0 0 8 * * ?")	按 cron 表达式 定点触发

六、运行与验证

```
mvn spring-boot:run
```

控制台预期输出：

```
天气已更新：[19:35:00] 北京 - 温度：22.3°C，风速：12.5 km/h  
[fixedDelay] 任务执行：19:35:05（上次执行结束后等待 5 秒）  
[fixedRate] 任务执行：19:35:06（每 6 秒执行一次，不考虑上次是否完成）
```

生日邮件任务默认每天早上 8:00 触发，调试时可临时修改 cron 为 `0 * * * * ?`（每分钟执行）。